Máster en Tecnologías del Lenguaje
Técnicas Basadas en Grafos Aplicadas al Procesamiento del Lenguaje Natural

Tarea opcional del tema 5: Instrucciones del programa palabras_clave-lem.py

Ariadna Frutos Javierre
15 de junio de 2020

Índice

Índice 1
1. Introducción 2
2. Requerimientos 2
3. Uso del programa 2
   3.0. Generación del grafo Pajek 2
   3.1. Inicio del programa 3
   3.2. Salida 4
   3.3. Ejecución del programa 4
4. Bibliografía 7

1. Introducción

El programa palabras_clave-lem.py, implementado en Python, identifica las palabras clave de una serie de documentos utilizando el método propuesto en [Mihalcea 2004]. Este método consiste en aplicar el algoritmo TextRank a un grafo cuyos nodos son las palabras del texto. Entre los nodos existe un enlace si los términos ocurren en el texto a una distancia menor que la definida por una ventana a determinar por el usuario. El peso de los enlaces debe proporcionarlo el usuario mediante un fichero en formato Pajek [Pajek 1998].

En caso de no disponer de un grafo de referencia, este puede generarse a partir del conjunto de documentos a analizar utilizando el programa coocurrencias-lem.py, incluido en este paquete. Para instrucciones sobre el programa coocurrencias-lem.py, véase [Frutos 2020][1].

2. Requerimientos

Los sistemas operativos admitidos son Windows y Linux. Se requiere tener Python instalado, así como los siguientes módulos:

- os
- getopt
- sys
- tkinter
- glob
- nltk
- re
- datetime

3. Uso del programa

3.0. Generación del grafo Pajek

En caso de no disponer de un grafo en formato Pajek que incluya los términos de los documentos a analizar y el peso sus enlaces, antes de proceder a la identificación de palabras clave es necesario generarlo. Esto puede hacerse utilizando el programa coocurrencias-lem.py. Para ello debe ejecutarse la siguiente orden:

pythonr *ruta_al_programa*/coocurrencias-lem.py -i <*ruta_carpeta_entrada*> -o <*ruta_carpeta_salida*> -w <*ventana*> -t <*etiquetas*>

Para información detallada sobre los argumentos incluidos, véase [Frutos 2020]. Si no se dispone de un corpus muy completo de documentos para generar el grafo, se recomienda utilizar la misma colección de los documentos a analizar; de otro modo, en el grafo faltarán términos y enlaces importantes para el análisis.

> NOTA: Para la generación del grafo que se incluye en el ejercicio, 2020-06-14_20-42-53_Grafo_Ventana_5_Etiquetas_['NN', 'NNS', 'JJ', 'JJR', 'JJS'].txt, se ha utilizado la colección etiquetada proporcionada por el profesorado, Archivos_Etiquetados, considerando una ventana de coocurrencia de 5 términos:

```
C:\Users\Usuario\Documents\coocurrencias>coocurrencias-lem.py -w 5 -i Archivos_Etiquetados -o grafos
```

[1] El programa coocurrencias-lem.py es igual al programa descrito en [Frutos 2020], coocurrencias-lem.py, con la única diferencia que en el primero, en lugar de truncar los términos, se lematizan.

```
Grafo generado
****************************************
Ruta del grafo: grafos/2020-06-14_20-42-53_Grafo_Ventana_5_Etiquetas_['NN', 'NNS', 'JJ', 'JJR', 'JJS'].txt
Número de vértices: 11758
Número de enlaces: 626461
```

3.1. Inicio del programa

El programa debe iniciarse desde la línea de comandos utilizando la siguiente orden:

python *ruta_al_programa*/palabras_clave-lem.py -i <*ruta_carpeta_entrada*> -o <*ruta_carpeta_salida*> -g <ruta_del_grafo_Pajek> -w <*ventana*> -d <*factor de amortiguación*> -l <*límite de convergencia*> -t <*etiquetas*>

Si no se incluyen los argumentos de entrada al llamar el programa, éste los requerirá explícitamente o utilizará los fijados por defecto. Para la selección de la carpeta donde se ubican los archivos de entrada y para la selección del fichero correspondiente al grafo Pajek, si el usuario no los incluye al principio el programa abrirá el explorador de archivos del cliente para que pueda seleccionarlos una vez iniciado.

Los argumentos de entrada son:

-h, --help: Consulta de la ayuda

-i --input_path: Ruta de la carpeta de los documentos a analizar. Si no se informa, el programa abrirá el explorador de archivos.

Los documentos deben ser textos etiquetados mediante el formalismo término/etiqueta. Por ejemplo:

A/DT challenging/JJ problem/NN faced/VBN by/IN researchers/NNS and/CC developers/NNS

Para obtener textos etiquetados en este formato, puede utilizarse el etiquetador de Standford, http://nlp.stanford.edu/software/tagger.shtml.

-o, --output: Ruta de la carpeta donde se guardarán los archivos con las palabras clave. Si no se informa, el programa creará la carpeta “output” en el directorio donde se encuentre el fichero palabras_clave-lem.py y los guardará allí.

-g, --graph: Ruta del fichero donde se encuentra el fichero de texto en formato Pajek con los pesos de los enlaces.

-w, --window Ventana de co-ocurrencia. Debe ser un número entero. El programa lo utilizará para definir la ventana de términos entre los que debe darse entre una pareja de términos para que se considere que co-ocurren.

-d, --damp Factor de corrección (*damping factor*). Debe ser un número entre 0 y 1, sin incluir el 0. En [Mihalcea 2004] se recomienda asignar d = 0,85.

-l, --threshold Límite de convergencia (threshold). Debe ser un número decimal. En [Mihalcea 2004] se asigna l = 0,0001.

-t, --tags: Listado de etiquetas del Penn Treebank [Marcus 1993] aceptadas como nodos del grafo. Por ejemplo:

["NN", "NNS", "JJ", "JJR", "JJS"].

Si no se informa, o si se informa con la cadena "all", se incluirán todas las etiquetas (no se filtrará).

Las etiquetas del Penn Treebank pueden consultarse en https://www.ling.upenn.edu/courses/Fall_2003/ling001/penn_treebank_pos.html.

Es importante filtrar, como mínimo, por las mismas etiquetas con que se han filtrado los términos del grafo de pesos, ya que si se aceptan más categorías que allí habrá términos inexistentes en el grafo y el programa generará un error. También se recomienda, por coherencia, utilizar el mismo valor de la ventana de términos, aunque no es obligatorio.

3.2. Salida

La salida es un conjunto de documentos, uno para cada documento del conjunto original, con entre 5 y 20 palabras clave del texto correspondiente. Los nombres de los ficheros de salida coinciden con el del fichero de entrada, con el término ".result" añadido. Es decir, a un fichero de entrada con nombre "fichero1.txt" le corresponde el fichero de salida "fichero1.txt.result".

También se genera un fichero resumen, "summary.csv", con la siguiente información, para cada documento procesado:

- Nombre del fichero original
- Nombre del fichero final
- Valor la ventana
- Valor del factor de amortiguación
- Valor del límite de convergencia
- Suma de puntuaciones finales
- Convergencia (diferencia entre las sumas de puntuaciones obtenidas en las dos últimas iteraciones)
- Número de iteraciones realizadas

3.3. Ejecución del programa

A continuación se describen los pasos que sigue el programa y la información que muestra por pantalla. Para pasar de una etapa a otra, el usuario debe confirmar la importación de datos cuando el programa lo requiere mediante la tecla "Enter".

El programa empieza anotando por pantalla los parámetros de entrada seleccionados por el usuario:

```
C:\Users\Usuario\Documents\coocurrencias>palabras_clave-lem.py -w 5 -d 0.85 -l 0.0001 -i Archivos_Etiquetados -o salida -
g "grafos/2020-06-14_20-42-53_Grafo_Ventana_5_Etiquetas_['NN', 'NNS', 'JJ', 'JJR', 'JJS'].txt"


La red se calculará utilizando los siguientes valores:
*****************************************
Ruta de los documentos a analizar: Archivos_Etiquetados
Fichero con el grafo de entrada: grafos/2020-06-14_20-42-53_Grafo_Ventana_5_Etiquetas_['NN', 'NNS', 'JJ', 'JJR', 'JJS'].t
xt
Ruta de salida: salida/
Ventana de terminos para considerar co-ocurrencia: 5
Etiquetas aceptadas: ['NN', 'NNS', 'JJ', 'JJR', 'JJS']
Factor de correccion: 0.85
Límite de convergencia (threshold) l: 0.0001
*****************************************

Pulsa enter para continuar
```

Una vez confirmados los datos, se extraen los índices de los vértices y los pesos del grafo

```
Extrayendo el vocabulario y el peso de los enlaces...


Datos del grafo del grafo importado:
*****************************************
Número de nodos: 11758
Número de enlaces: 626461

Pulsa enter para continuar
```

El tercer paso es importar los textos a analizar. El programa muestra por pantalla el título de los ficheros importados y los primeros 50 primeros caracteres del texto que contienen:

```
Importando textos...

144 textos importados:
*****************************************
'Documento0: \C-41.txt.tagged' '<s> A/DT challenging/JJ problem/NN faced/VBN by/IN...'
'Documento1: \C-42.txt.tagged' '<s> Ensemble/NNP Kalman/NNP filter/NN data/NNS ass...'
'Documento2: \C-44.txt.tagged' '<s> Wireless/NNP Sensor/NNP Networks/NNP have/VBP ...'
'Documento3: \C-45.txt.tagged' '<s> The/DT problem/NN of/IN localization/NN in/IN ...'
'Documento4: \C-46.txt.tagged' '<s> Archival/JJ storage/NN of/IN sensor/NN data/NN...'
'Documento5: \C-48.txt.tagged' '<s> In/IN many/JJ sensor/NN networks/NNS
data/NNS...'
```

```
'Documento140: \J-71.txt.tagged' '<s> I/PRP develop/VBP a/DT new/JJ mechanism/NN for...'
'Documento141: \J-72.txt.tagged' '<s> We/PRP consider/VBP the/DT parallels/NNS betwe...'
'Documento142: \J-73.txt.tagged' '<s> We/PRP introduce/VBP new/JJ online/NN models/N...'
'Documento143: \J-74.txt.tagged' '<s> Motivated/VBN by/IN the/DT rise/NN of/IN onlin...'
*****************************************

Pulsa enter para continuar
```

Tras importar los textos, la aplicación los procesa uno a uno, hasta generar el fichero de salida con las palabras clave. Se empieza por generar listas de tuplas (término, etiqueta), las cuales se normalizan y filtran de forma análoga coocurrencias-lem.py (véase [Frutos 2020]). Para el lematizado, que sustituye al truncado realizado en [Frutos 2020] tanto en coocurrencias-lem.py como en palabras_clave-lem.py, se utiliza el módulo WordNetLemmatizer de la librería NLTK. El resultado de este procesamiento es una lista con los términos que se tendrán en cuenta para la identificación de palabras clave. La aplicación muestra los primeros y los últimos términos del texto:

```
Normalización de textos y calculo de palabras clave...

*****************************************
```

Por ejemplo, para el texto 2 el resultado es:

```
TEXTO 2 : \C-44.txt.tagged

Términos del documento: [['use', 'many', 'locationdependent', 'application', 'need', 'location', 'wireless', '  ...  ', '
one', 'msp', 'scanning', 'foot', 'foot', 'foot']]
```

Esta lista incluye los términos tal como se dan en el texto, con la misma ordenación y con repeticiones. A continuación, los términos son transformados en números, según la indexación proporcionada por el grafo de referencia:

```
Nodos ordenados del documento: ( 3506 en total) 2 : [[11104, 5898, 5674, 499, 6595, 5669, 11504, '  ...  ', 7076, 6412, 9
060, 3758, 3758, 3758]]
```

Nótese la correspondencia entre los términos y los números de las dos últimas imágenes: es la correspondencia marcada por el grafo de referencia, menos uno:

*Vertices 11758
:
:
5675 "locationdependent"
:
:
5899 "many"
:
:
11105 "use"
:
:

La unidad de menos es debido a la indexación de listas en Python, que empieza en cero. En la aplicación, pues, al vértice 1 del grafo de referencia se le asigna el índice 0, y así sucesivamente.

A partir de la lista de términos se genera la lista de nodos únicos que constituirán el grafo del documento, así como las parejas de nodos que coocurren dentro de la ventana seleccionada. Los primeros y últimos nodos y parejas se muestran por pantalla:

```
Nodos únicos del documento: ( 663 en total) 2 : [[11, 24, 30, 60, 77, 79, 83, '  ...  ', 11466, 11504, 11535, 11540, 1158
3, 11584]]

Parejas del documento: [[(11104, 5898), (11104, 5674), (11104, 499), (11104, 6595), (5898, 5674), (5898, 499), (5898, 659
5), (5898, 5669), (5674, 499), (5674, 6595), '  ...  ', (6412, 7076), (7486, 9060), (7486, 7076), (9060, 7076), (9060, 90
60), (9060, 3758), (7076, 6412), (7076, 9060), (7076, 3758), (3758, 3758)]]
```

Con esta información ya puede aplicarse el algoritmo iterativo TextRank. Cuando se alcanza la convergencia determinada por el parámetro threshold, se ordenan las puntuaciones y se identifican las palabras correspondientes, las cuales se exportan a un fichero de texto junto con las puntuaciones adquiridas. Se muestran los datos informativos del rendimiento del algoritmo (suma de puntuaciones final, convergencia, número de iteraciones), las palabras clave y sus respectivas puntuaciones, y la ruta del fichero de salida.

```
Parejas del documento: [[(11104, 5898), (11104, 5674), (11104, 499), (11104, 6595), (5898, 5674), (5898, 499), (5898, 659
5), (5898, 5669), (5674, 499), (5674, 6595), '  ...  ', (6412, 7076), (7486, 9060), (7486, 7076), (9060, 7076), (9060, 90
60), (9060, 3758), (7076, 6412), (7076, 9060), (7076, 3758), (3758, 3758)]]

Cálculo de puntuaciones: iteraciones TextRank...

Suma de puntuaciones final:  658.7497555522809 ; Diferencia:  9.269645465792564e-05 ; Threshold:  0.0001 ; Verdad:  False
 Número de iteraciones: 26
Palabras clave:  {'node': 53.587607867641964, 'figure': 14.451853994147454, 'event': 13.79959230972476, 'time': 13.212517
171643599, 'number': 13.004308678060942, 'e': 11.919275083160286, 'sensor': 11.545202272798377, 'system': 10.389727639531
047, 'section': 9.952840979971983, 'b': 9.760699513406145, 'sequence': 9.286284540776107, 'result': 8.94575834005207, 'lo
cation': 8.802415286022278, 'information': 8.763161409505248, 'n': 8.40528062339373, 'c': 8.131990914185074, 'g': 7.99370
7396323708, 'msp': 7.718566566329261, 'network': 7.4717947995251555, 'algorithm': 7.461438487342301, 'j': 6.5107638738087
1}
Palabras clave almacenadas en el fichero:  salida/\C-44.txt.tagged.result
```

El programa finaliza informando del hecho por pantalla y recordando la existencia del fichero summary.csv, por si se quieren analizar los resultados de la ejecución.

4. Bibliografía

[Frutos 2020] Frutos, A. (2020), *Tarea del tema 5: Instrucciones del programa palabras_clave-lem.py*, trabajo presentado en la asignatura "Técnicas Basadas en Grafos Aplicadas al Procesamiento del Lenguaje Natural" del Máster en Tecnologías del Lenguaje de la UNED.

[Mihalcea 2004] Mihalcea, R y Tarau, P. (2004). *TextRank: Bringing Order into Texts.* Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing

[Pajek 1998], Batagelj, V. y MrvarPajek, A. (1998). *Program for Large Network Analysis*, Connections 21, p. 47. Para una descripción del formato Pajek, http://vlado.fmf.uni-lj.si/pub/networks/pajek/howto/text2pajek.htm